

MiniANN: C++ Artificial Neural Network Library

System Architecture & Software Engineering Specification

Version 1.3 · Problem Statement 5 · Modern C++17 Standard · Five-Member Engineering Distribution

Member	Assigned Module	Primary Domain Responsibilities	Extended ables	Deliver- ables
M1	Network Core	Layer, NeuralNetwork, Topologies, Validation	Architecture	Skeletons,
M2	Neuron & Activation	Neuron, IActivation, Polymorphic Transducers	Multi-layer chaining	Xavier/He Weight Initializers
M3	Training & Optimization	Trainer, ILoss, Backpropagation Engine	Mini-batch	Training,
M4	Data & Evaluation	Dataset, Preprocessing, Core Metrics	Adam Optimizer	CSV Ingestion Engine,
M5	Integration & Storage	ModelSerializer, Logging Framework, CLI Runners	Confusion Matrix	CSV Loss Exporter,
				Demo Applications

1. Overview & Design Philosophy

MiniANN is a modular, lightweight feed-forward Artificial Neural Network (ANN) framework engineered from scratch in modern C++17. As explicitly defined in Problem Statement 5, **architectural clarity, strict encapsulation, and clean Object-Oriented Design (OOD) supersede computational execution speed.**

The system exposes clean interfaces to construct custom layer topologies, train on benchmark datasets using analytical gradient descent, evaluate classifications with statistical metrics, and persist trained models to disk. High-level production frameworks (e.g., PyTorch, TensorFlow) and black-box matrix acceleration libraries (e.g., Eigen, BLAS) are deliberately excluded to illuminate foundational mechanics:

- **Abstraction:** Mathematical concepts (activations, loss calculations, optimization routines) are declared as pure virtual interfaces (`IActivation`, `ILoss`, `IOptimizer`, `IMetric`).
- **Encapsulation:** Neuronal parameters ($\mathbf{w}$, b) and transient computational states ($\mathbf{z}$, δ) are strictly private, accessible solely via immutable getters or explicit stepping methods.
- **Composition over Inheritance:** A `NeuralNetwork` is composed of `Layer` instances, which in turn encapsulate `Neuron` primitives.
- **Separation of Responsibilities:** The network owns forward prediction and gradient propagation. It does not train itself; training policies and parameter mutations are strictly governed by an external `Trainer` and `IOptimizer`.

2. Mathematical Foundations & Propagation Dynamics

2.1 Forward Dynamics

For a single neuron receiving an input activation vector $\mathbf{x} \in \mathbb{R}^n$ with synaptic weight vector $\mathbf{w} \in \mathbb{R}^n$ and scalar bias $b \in \mathbb{R}$:

$$z = \sum_{i=1}^n w_i x_i + b = \mathbf{w}^T \mathbf{x} + b, \quad a = f(z) \quad (1)$$

where z represents the affine pre-activation sum and $f(\cdot)$ is the non-linear transfer function. In layer-wise matrix notation for layer l containing m neurons receiving inputs $\mathbf{a}^{[l-1]} \in \mathbb{R}^n$:

$$\mathbf{z}^{[l]} = \mathbf{W}^{[l]} \mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}, \quad \mathbf{a}^{[l]} = f^{[l]}(\mathbf{z}^{[l]}) \quad (2)$$

where $\mathbf{W}^{[l]} \in \mathbb{R}^{m \times n}$ and $\mathbf{b}^{[l]} \in \mathbb{R}^m$.

2.2 Activation Function Derivatives

All derivatives are evaluated analytically with respect to the pre-activation input z :

- **Sigmoid:** $f(z) = \frac{1}{1+e^{-z}}$, with derivative $f'(z) = f(z) \cdot (1 - f(z))$.
- **Hyperbolic Tangent (Tanh):** $f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$, with derivative $f'(z) = 1 - f(z)^2$.
- **Rectified Linear Unit (ReLU):** $f(z) = \max(0, z)$, with subgradient $f'(z) = 1$ if $z > 0$, else 0.

2.3 Backpropagation via the Analytical Chain Rule

Given the Mean Squared Error (MSE) loss function for a single training exemplar with output dimension K :

$$\mathcal{L} = \frac{1}{K} \sum_{k=1}^K (\hat{y}_k - y_k)^2 \implies \frac{\partial \mathcal{L}}{\partial \hat{y}_k} = \frac{2}{K} (\hat{y}_k - y_k) \quad (3)$$

For any neuron j in layer l , define the local error sensitivity $\delta_j^{[l]} \equiv \frac{\partial \mathcal{L}}{\partial z_j^{[l]}}$:

$$\delta_j^{[L]} = \frac{\partial \mathcal{L}}{\partial a_j^{[L]}} \cdot f' \left(z_j^{[L]} \right) \quad (\text{Output Layer}), \quad \delta_j^{[l]} = \left(\sum_{k=1}^{M^{[l+1]}} w_{kj}^{[l+1]} \delta_k^{[l+1]} \right) \cdot f' \left(z_j^{[l]} \right) \quad (\text{Hidden Layer}) \quad (4)$$

The parameter gradients accumulate across samples as follows:

$$\frac{\partial \mathcal{L}}{\partial w_{ji}^{[l]}} = \delta_j^{[l]} \cdot a_i^{[l-1]}, \quad \frac{\partial \mathcal{L}}{\partial b_j^{[l]}} = \delta_j^{[l]}, \quad \frac{\partial \mathcal{L}}{\partial a_i^{[l-1]}} = \sum_{j=1}^{M^{[l]}} w_{ji}^{[l]} \delta_j^{[l]} \quad (5)$$

2.4 Optimization Dynamics: SGD and Adam

In standard Stochastic Gradient Descent (SGD) with learning rate η , parameters update along the negative batch-averaged gradient:

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla_{\mathbf{W}} \mathcal{L}, \quad \mathbf{b} \leftarrow \mathbf{b} - \eta \nabla_{\mathbf{b}} \mathcal{L} \quad (6)$$

For the **Adam Optimizer** (Adaptive Moment Estimation), running estimates of the first moment (mean m_t) and uncentered second moment (variance v_t) of the gradient g_t are tracked at step t ($\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (7)$$

Correcting for initialization bias towards zero:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \implies \theta_t \leftarrow \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \quad (8)$$

2.5 Weight Initialization Formulations

Zero-initialization prevents symmetry breaking, causing all neurons in a layer to compute identical updates. To preserve activation and gradient variance through deep topologies:

- **Xavier / Glorot Initialization** (for Sigmoid and Tanh activations):

$$w \sim \mathcal{U} \left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, +\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}} \right) \quad (9)$$

- **He / Kaiming Initialization** (for ReLU activations):

$$w \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\text{in}}}}\right) \quad (10)$$

3. System Architecture Diagram

The diagram below illustrates the decoupled structure of MiniANN, highlighting clear ownership boundaries, composition, and polymorphic strategy abstractions.

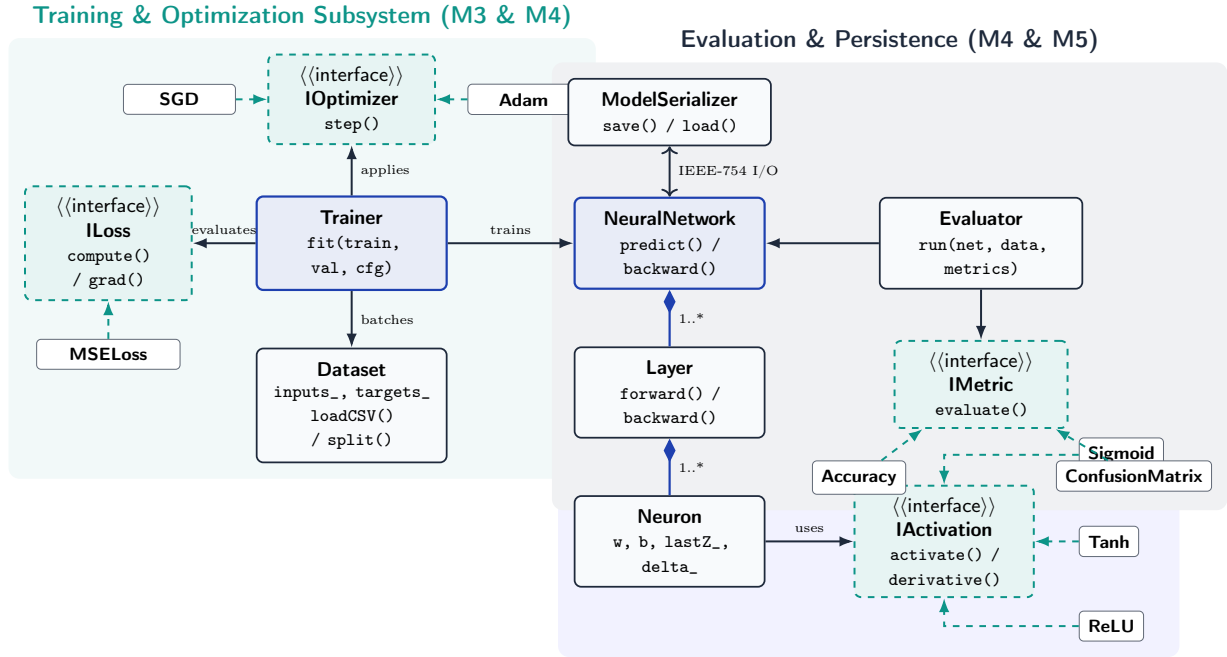


Figure 1: MiniANN Class Relationship & Architectural Component Hierarchy.

4. Execution Workflow & Propagation Pipeline

The execution lifecycle of MiniANN enforces a clean unidirectional computational pipeline. The model forward pass calculates and caches states; the loss computes outer sensitivities; the backward pass propagates gradients; and the optimizer executes parameter steps.

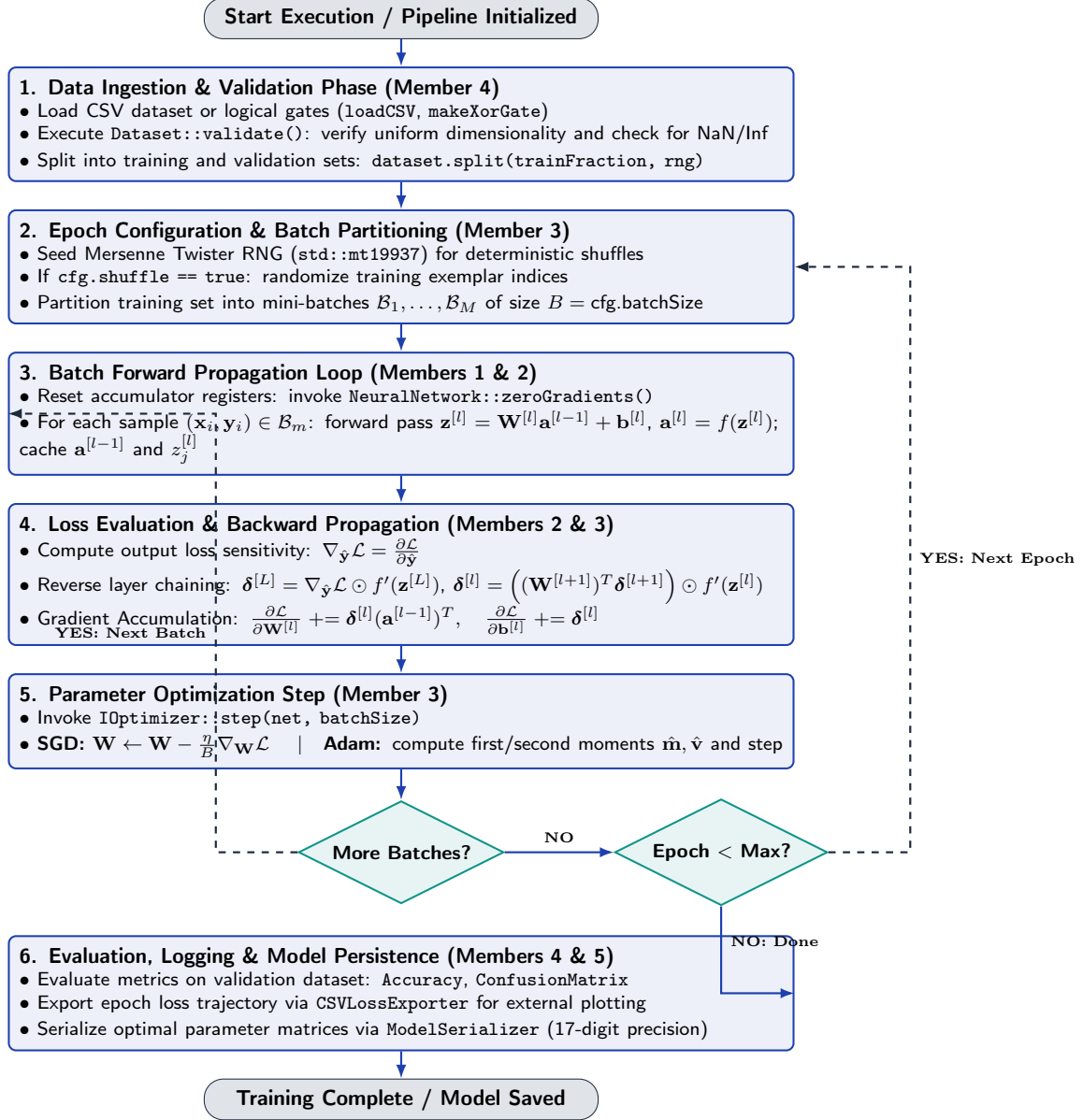


Figure 2: End-to-End Computational Flow and Gradient Propagation Pipeline.

5. Concrete Interface Specifications

All core types reside in namespace `miniann`. Headers are located in `include/miniann/`. These class definitions form the formal binding contract between all 5 team members.

5.1 Common Types & Activation Subsystem (types.hpp, activation.hpp – Member 2)

```
#pragma once
#include <vector>
#include <string>
#include <memory>
#include <cmath>
#include <stdexcept>

namespace miniann {
using Vector = std::vector<double>;
using Matrix = std::vector<std::vector<double>>;

class IActivation {
public:
    virtual ~IActivation() = default;
    virtual double activate(double z) const = 0;
    virtual double derivative(double z) const = 0; // Evaluated at pre-activation z
    virtual std::string name() const = 0;
};
using ActivationPtr = std::shared_ptr<const IActivation>;

class Sigmoid : public IActivation {
public:
    double activate(double z) const override { return 1.0 / (1.0 + std::exp(-z)); }
    double derivative(double z) const override {
        double a = activate(z);
        return a * (1.0 - a);
    }
    std::string name() const override { return "sigmoid"; }
};

class Tanh : public IActivation {
public:
    double activate(double z) const override { return std::tanh(z); }
    double derivative(double z) const override {
        double a = std::tanh(z);
        return 1.0 - (a * a);
    }
    std::string name() const override { return "tanh"; }
};

class ReLU : public IActivation {
public:
    double activate(double z) const override { return z > 0.0 ? z : 0.0; }
    double derivative(double z) const override { return z > 0.0 ? 1.0 : 0.0; }
    std::string name() const override { return "relu"; }
};

class ActivationFactory {
public:
    static ActivationPtr create(const std::string& name); // Throws std::invalid_argument
};
} // namespace miniann
```

5.2 Neuron Primitive & Parameter Management (neuron.hpp – Member 2)

```
#pragma once
#include "miniann/types.hpp"
#include "miniann/activation.hpp"
#include <random>

namespace miniann {
enum class WeightInit { Uniform, Xavier, He };
}
```

```

class Neuron {
public:
    Neuron(std::size_t numInputs, std::size_t numOutputs, ActivationPtr act,
           std::mt19937& rng, WeightInit initMethod = WeightInit::Xavier);

    double forward(const Vector& inputs);
    void backward(double dLoss_dOutput); // Accumulates dL/dw, sets delta_

    double delta() const { return delta_; }
    const Vector& weights() const { return weights_; }
    double bias() const { return bias_; }
    const Vector& gradWeights() const { return gradWeights_; }
    double gradBias() const { return gradBias_; }
    const IActivation& activation() const { return *activation_; }

    void setParameters(Vector w, double b); // Used by ModelSerializer
    void applyStep(const Vector& dW, double dB); // w += dW, b += dB (used by optimizers)
    void zeroGradients();

private:
    Vector weights_;
    double bias_;
    Vector gradWeights_;
    double gradBias_;
    ActivationPtr activation_;
    Vector lastInputs_;
    double lastZ_ = 0.0;
    double delta_ = 0.0;
};
} // namespace miniann

```

5.3 Layer & Network Composition (layer.hpp, network.hpp – Member 1)

```

#pragma once
#include "miniann/neuron.hpp"

namespace miniann {
class Layer {
public:
    Layer(std::size_t numNeurons, std::size_t numInputs, ActivationPtr act,
          std::mt19937& rng, WeightInit init = WeightInit::Xavier);

    Vector forward(const Vector& inputs);
    Vector backward(const Vector& dLoss_dOutput); // Returns dLoss/dInputs to propagate down
    void zeroGradients();

    std::size_t size() const { return neurons_.size(); }
    std::size_t inputSize() const { return inputSize_; }
    std::vector<Neuron>& neurons() { return neurons_; }
    const std::vector<Neuron>& neurons() const { return neurons_; }

private:
    std::vector<Neuron> neurons_;
    std::size_t inputSize_;
};

class NeuralNetwork {
public:
    void addLayer(Layer layer); // Throws std::invalid_argument if dimensions mismatch
    Vector predict(const Vector& input); // Non-const: caches intermediate z and activations
    void backward(const Vector& dLoss_dOutput);
    void zeroGradients();

    std::size_t numLayers() const { return layers_.size(); }
    std::vector<Layer>& layers() { return layers_; }
    const std::vector<Layer>& layers() const { return layers_; }

private:
    std::vector<Layer> layers_;
};
} // namespace miniann

```

5.4 Loss Functions, Optimizers & Trainer (loss.hpp, optimizer.hpp, trainer.hpp – Member 3)

```
#pragma once
#include "miniann/network.hpp"
#include "miniann/dataset.hpp"
#include "miniann/logger.hpp"

namespace miniann {
class ILoss {
public:
    virtual ~ILoss() = default;
    virtual double compute(const Vector& predicted, const Vector& target) const = 0;
    virtual Vector gradient(const Vector& predicted, const Vector& target) const = 0;
};

class MSELoss : public ILoss {
public:
    double compute(const Vector& predicted, const Vector& target) const override;
    Vector gradient(const Vector& predicted, const Vector& target) const override;
};

class IOptimizer {
public:
    virtual ~IOptimizer() = default;
    virtual void step(NeuralNetwork& net, std::size_t batchSize) = 0;
};

class SGD : public IOptimizer {
private:
    double lr_;
public:
    explicit SGD(double learningRate) : lr_(learningRate) {}
    void step(NeuralNetwork& net, std::size_t batchSize) override;
};

class Adam : public IOptimizer {
private:
    double lr_, beta1_, beta2_, eps_;
    std::size_t t_ = 0;
    std::vector<std::vector<Vector>> m_w_, v_w_;
    std::vector<std::vector<double>> m_b_, v_b_;
public:
    explicit Adam(double lr = 0.001, double b1 = 0.9, double b2 = 0.999, double eps = 1e-8);
    void step(NeuralNetwork& net, std::size_t batchSize) override;
};

struct TrainingConfig {
    int epochs = 1000;
    std::size_t batchSize = 1; // 1 = Online SGD, >1 = Mini-batch, 0 = Full Batch
    bool shuffle = true;
    unsigned seed = 42;
    int logEvery = 100;
};

struct TrainingHistory {
    std::vector<double> trainLoss;
    std::vector<double> validationLoss;
};

class Trainer {
public:
    Trainer(NeuralNetwork& net, const ILoss& loss, IOptimizer& opt, ILogger* logger = nullptr)
        ;
    TrainingHistory fit(const Dataset& train, const Dataset* validation, const TrainingConfig&
        cfg);
private:
    NeuralNetwork& net_;
    const ILoss& loss_;
    IOptimizer& opt_;
    ILogger* logger_;
};
} // namespace miniann
```


5.5 Dataset Engine & Evaluation Metrics (dataset.hpp, metrics.hpp – Member 4)

```
#pragma once
#include "miniann/types.hpp"
#include <string>
#include <random>
#include <map>

namespace miniann {
class Dataset {
public:
    void add(Vector input, Vector target);
    std::size_t size() const { return inputs_.size(); }
    const Vector& input(std::size_t i) const { return inputs_.at(i); }
    const Vector& target(std::size_t i) const { return targets_.at(i); }
    void shuffle(std::mt19937& rng);
    std::pair<Dataset, Dataset> split(double trainFraction, std::mt19937& rng) const;
    void validate() const; // Throws on empty set, dimension mismatch, or NaN/Inf values

    static Dataset loadCSV(const std::string& path, bool hasHeader, int targetColIndex, int
        targetDim);
private:
    std::vector<Vector> inputs_;
    std::vector<Vector> targets_;
};

Dataset makeAndGate();
Dataset makeOrGate();
Dataset makeXorGate();

class IMetric {
public:
    virtual ~IMetric() = default;
    virtual double evaluate(const std::vector<Vector>& preds,
        const std::vector<Vector>& targets) const = 0;
    virtual std::string name() const = 0;
};

class Accuracy : public IMetric {
public:
    double evaluate(const std::vector<Vector>& preds,
        const std::vector<Vector>& targets) const override;
    std::string name() const override { return "accuracy"; }
};

class ConfusionMatrix : public IMetric {
public:
    explicit ConfusionMatrix(std::size_t numClasses) : numClasses_(numClasses) {}
    double evaluate(const std::vector<Vector>& preds,
        const std::vector<Vector>& targets) const override;
    std::string name() const override { return "confusion_matrix"; }
    void print() const;
    double precision(std::size_t classIdx) const;
    double recall(std::size_t classIdx) const;
private:
    std::size_t numClasses_;
    mutable std::vector<std::vector<std::size_t>> matrix_;
};
} // namespace miniann
```

5.6 Logging, Model Persistence & Visualization (logger.hpp, serializer.hpp – Member 5)

```
#pragma once
#include "miniann/network.hpp"
#include "miniann/trainer.hpp"
#include <string>

namespace miniann {
enum class LogLevel { Debug, Info, Warn, Error };

class ILogger {
public:
    virtual ~ILogger() = default;
    virtual void log(LogLevel level, const std::string& message) = 0;
};
```

```
};

class ConsoleLogger : public ILogger {
public:
    void log(LogLevel level, const std::string& message) override;
};

class ModelSerializer {
public:
    static void save(const NeuralNetwork& net, const std::string& path);
    static NeuralNetwork load(const std::string& path, std::mt19937& rng);
};

class CSVLossExporter {
public:
    static void exportHistory(const TrainingHistory& hist, const std::string& filepath);
};
} // namespace miniann
```

6. Mini-Batch Execution Algorithm

The updated execution sequence integrates dynamic batch segmentation and gradient accumulation:

1. **Validation:** Invoke `train.validate()`. Instantiate RNG from `cfg.seed`.
2. **Epoch Loop:** For $\text{epoch} \in [1, \dots, \text{cfg.epochs}]$:
 - Shuffle local training copy if `cfg.shuffle == true`.
 - Determine batch size $B = (\text{cfg.batchSize} == 0) ? \text{train.size()} : \text{cfg.batchSize}$.
 - Subdivide data into batches $\mathcal{B}_1, \mathcal{B}_2, \dots, \mathcal{B}_M$.
 - For each mini-batch $\mathcal{B}_m$:
 - (a) Execute `net.zeroGradients()`.
 - (b) For each exemplar $(\mathbf{x}_i, \mathbf{y}_i) \in \mathcal{B}_m$:
 - Forward: $\hat{\mathbf{y}}_i = \text{net.predict}(\mathbf{x}_i)$ (caches $\mathbf{z}$ and inputs in neurons).
 - Evaluate Loss: $\ell_i = \text{loss.compute}(\hat{\mathbf{y}}_i, \mathbf{y}_i)$.
 - Backward: `net.backward(loss.gradient( $\hat{\mathbf{y}}_i, \mathbf{y}_i$ ))` (accumulates $\nabla_{\mathbf{w}} \mathcal{L}$).
 - (c) Parameter Update: `optimizer.step(net, batchSize)` (applies averaged update).
 - Append epoch mean training loss $\bar{\mathcal{L}}_{\text{train}}$ and validation loss $\bar{\mathcal{L}}_{\text{val}}$ to `TrainingHistory`.
 - If $\text{epoch} \pmod{\text{cfg.logEvery}} == 0$: Emit formatted logging status.

7. Architectural Rationale & OOP Pattern Mapping

8. Five-Member Engineering Distribution & Integration Seams

Critical Integration Seams (Strict Team Consensus Required):

1. **Pre-activation Evaluation:** `derivative(z)` receives z (pre-activation), not activation output a (M2 $\leftrightarrow$ M3).
2. **Accumulation Semantics:** `Neuron::backward` accumulates gradients ($+=$) instead of overwriting ($=$) to enable batching (M2 $\leftrightarrow$ M3).
3. **Unmodified Weights in Error Chaining:** `Layer::backward` computes $\frac{\partial \mathcal{L}}{\partial a}$ using synaptic weights prior to the optimizer stepping phase (M1 $\leftrightarrow$ M3).
4. **Grammar Synchronization:** Serializer string tags must match `ActivationFactory::create` naming tokens exactly (M2 $\leftrightarrow$ M5).

#	Architectural Decision	Engineering Rationale	Alternative Evaluated
D1	Shared Stateless Activations via <code>shared_ptr</code>	Allows identical activation instances across all neurons in a layer without duplication or heap reallocation.	Prototype Pattern with <code>clone()</code> method.
D2	Decoupled Trainer Strategy	Decouples forward propagation topology from optimization algorithms (Strategy Pattern).	Network-owned internal <code>train()</code> method.
D3	Pre-Activation Gradient Derivations	Prevents silent calculation defects when evaluating piecewise linear transfers such as ReLU.	Deriving directly from post-activation <i>a</i> .
D4	In-Neuron Gradient Accumulation	Enables batching naturally: accumulates $\sum \nabla \mathcal{L}$ across <i>B</i> samples before stepping.	Returning vectors from every <code>backward()</code> call.
D5	Native <code>std::vector</code> Over Matrix Libs	Emphasizes clear Object-Oriented component design over linear algebra optimizations.	Third-party tensor engine (Eigen / Armadillo).
D6	Strict C++ Standard Exceptions	Eliminates partially instantiated objects and unpropagated error return codes.	C-style integer return codes.
D7	Plain Text IEEE-754 Serialization	Ensures reproducible model persistence and human-inspectable debugging diffs.	Opaque binary serialization.
D8	Non-const <code>predict()</code> Method	Cached pre-activations and inputs inside neurons are required for backpropagation.	Passing state objects through calls.
D9	Optimizer Takes Network Reference	Permits stateful optimizers (e.g., Adam) to track per-weight momentum tables.	Per-neuron self-updating logic.

OOP Principle / Pattern	Concrete Manifestation in MiniANN Architecture
Abstraction	Pure virtual interfaces: <code>IActivation</code> , <code>ILoss</code> , <code>IOptimizer</code> , <code>IMetric</code> , <code>ILogger</code> .
Polymorphism	Dynamic dispatch across activations (<code>Sigmoid</code> , <code>Tanh</code> , <code>ReLU</code>) and optimizers (<code>SGD</code> , <code>Adam</code>).
Encapsulation	Strict private member variables (<code>weights_</code> , <code>bias_</code> , <code>lastZ_</code>) with constrained mutation accessors.
Composition	<code>NeuralNetwork</code> $\blacklozenge \longrightarrow$ <code>Layer</code> $\blacklozenge \longrightarrow$ <code>Neuron</code> , and <code>Trainer</code> collaborating with independent abstractions.
Strategy Pattern	Swappable optimization schemes (<code>IOptimizer</code>) and error formulations (<code>ILoss</code>).
Factory Pattern	Decoupled activation instantiation from string descriptors via <code>ActivationFactory</code> .

Member	Module Domain	Mandatory Core Deliverables	Extended Feature Assignment
M1	Network Core & Topology	Layer/Network composition, validation checks, input propagation.	Stub header definitions for team synchronization (Week 1).
M2	Neuron & Activation System	<code>Neuron</code> math, state caches, <code>Sigmoid</code> , <code>ReLU</code> , <code>Tanh</code> .	Xavier & He Initializer implementations.
M3	Training & Optimization	<code>ILoss</code> , <code>Trainer</code> fit loop, SGD, analytical Gradient Checker.	Mini-batch execution & Adam Optimizer.
M4	Data Pipeline & Metrics	<code>Dataset</code> normalization, splits, <code>Accuracy</code> , <code>MSEMetric</code> .	Robust CSV Loader & Multi-class Confusion Matrix.
M5	Storage, I/O & Integration	IEEE-754 Serializer, <code>ILogger</code> , CMake configs, AND/OR/XOR demos.	CSV Loss Exporter & Iris Multiclass CLI Demo.

9. Repository Layout & Coding Standards

```
MiniANN/
|-- CMakeLists.txt
|-- include/miniann/
|   |-- types.hpp           activation.hpp   neuron.hpp       layer.hpp
|   |-- network.hpp        loss.hpp       optimizer.hpp    trainer.hpp
|   |-- dataset.hpp        metrics.hpp    logger.hpp       serializer.hpp
|   '-- exporter.hpp
|-- src/                    (One matching .cpp per header implementation)
|-- tests/                 (One focused unit test suite per module)
|-- demos/
|   |-- xor_demo.cpp        and_or_demo.cpp iris_demo.cpp
|-- docs/                  (Architecture specifications and diagrams)
```

- **C++17 Standards:** Build with `-std=c++17 -Wall -Wextra -Wpedantic`. Zero raw memory operations (`new/delete`).
- **Const Correctness:** All non-mutating inspection methods are declared `const`.
- **Naming Conventions:** Classes use PascalCase, interfaces use IPascalCase, functions use camelCase, and private members take trailing underscores (`weights_`).

10. Verification, Quality Control & Gradient Check Protocol

Module / Target	Verification Procedure & Acceptance Criteria
Activations	Verify outputs against known points ($\sigma(0) = 0.5, \text{ReLU}(-2) = 0$). Verify derivative matches finite-difference approximation within 10^{-7} .
Neuron	Single-neuron forward and backward propagation match hand-calculated values on known weights.
Layer / Network	Dimensional validation: verify that dimension mismatches throw <code>std::invalid_argument</code> .
Gradient Check	Compare analytical gradients against centered finite difference approximations across all weights; relative error must be $< 10^{-6}$.
Optimizer	Verify that one parameter step with constant gradient yields mathematically exact weight displacement.
Dataset Engine	Split correctness: train/test split partitions cleanly without overlap. CSV parser handles ragged data.
Serializer	Persistence determinism: save network, reload into blank structure, verify $\max y_{\text{orig}} - y_{\text{loaded}} < 10^{-14}$.

10.1 Centered Difference Numerical Gradient Check Protocol

Because analytical backpropagation defects fail silently (causing poor convergence rather than compilation crashes), Member 3 owns the analytical gradient verification suite:

$$g_{\text{num}}(\theta) = \frac{\mathcal{L}(\theta + \epsilon) - \mathcal{L}(\theta - \epsilon)}{2\epsilon} \quad (11)$$

with perturbation $\epsilon = 10^{-5}$. The relative error metric must satisfy:

$$\frac{\|\mathbf{g}_{\text{num}} - \mathbf{g}_{\text{analytic}}\|_2}{\max(\|\mathbf{g}_{\text{num}}\|_2, \|\mathbf{g}_{\text{analytic}}\|_2) + 10^{-8}} < 10^{-6} \quad (12)$$

11. Model Serialization Specification

Trained networks are serialized as plain text with 17 digits of IEEE-754 precision (`std::setprecision(17)`):

```

MINIANN 1
layers 2
layer 4 2 tanh
0.15423671829102938 -0.92384719283746519 0.04918273645192837
0.81237461928374619 -0.49283746192837461 0.39182736451928374
0.27461928374619283 0.61928374619283746 -0.19283746192837461
-0.5827364519283746 0.12938475619283746 0.74619283746192837
layer 1 4 sigmoid
-0.81237461928374619 1.49283746192837461 -0.39182736451928374 0.82736451928374619
0.12938475619283746

```

- **Validation on Load:** Check magic string MINIANN, version integer, matching layer counts, and verify each neuron row contains exactly numInputs+1 floating-point values. Throw `std::runtime_error` on malformed files.

12. Development Roadmap & Milestones

Phase	Domain	Engineering Scope	Exit Criteria
1	Design	Freeze interface contracts, establish CMake harness and testing framework.	Compiling skeletons on all machines.
2	Core Logic	Implement activations, <code>Neuron</code> , <code>Layer</code> , and <code>NeuralNetwork::predict</code> .	Forward pass unit tests pass.
3	Optimization	Implement <code>MSELoss</code> , backpropagation chaining, SGD, and <code>Trainer</code> .	Gradient check passes ($< 10^{-6}$).
4	Data & Storage	Dataset CSV loader, evaluation metrics, logging, model serializer.	Round-trip persistence matches outputs.
5	Demonstration	Extended features (Adam, Confusion Matrix, CSV export) and demos.	End-to-end XOR and Iris demos pass.

13. Minimum Viable Product & Extended Deliverables Checklist

☐ Core MVP Deliverables:

- ☐ Feed-forward neural network composition (`NeuralNetwork` $\blacklozenge \rightarrow$ `Layer` $\blacklozenge \rightarrow$ `Neuron`).
- ☐ Polymorphic activation subsystem supporting Sigmoid, Tanh, and ReLU.
- ☐ Loss calculation and analytical gradient evaluation using Mean Squared Error.
- ☐ Backpropagation chaining with gradient accumulation and SGD optimization.
- ☐ Prediction pipeline caching intermediate forward values.
- ☐ Evaluation metric harness computing accuracy and MSE.
- ☐ Plain-text IEEE-754 model serialization and deserialization.
- ☐ Logging subsystem supporting console output.
- ☐ Verification demos learning AND, OR, and XOR truth tables.

☐ Extended Architecture Features:

- ☐ Mini-batch training support with dynamic batch segmentation.
- ☐ General-purpose RFC-4180 CSV dataset ingestion engine with header handling.
- ☐ Adam optimizer with first and second moment bias correction.
- ☐ Multi-class Confusion Matrix with per-class Precision and Recall reporting.
- ☐ Xavier/Glorot and He/Kaiming pseudorandom weight initializers.
- ☐ CSV training history exporter for loss curve plotting.

14. Risk Management & Mitigations

Identified Risk	Engineering Mitigation Strategy
Interface Drift During Development	Interfaces are frozen in Week 1. Any signature adjustment requires consensus from all 5 members.
Silent Backpropagation Defects	Centered difference numerical gradient check test suite must pass before any training tests begin.
Convergence Failure on Non-Linear Demos	Fixed pseudorandom seeds, Xavier initialization, and hidden topology sized with at least 4 neurons.
Integration Bottlenecks	Member 1 provides compiling header stubs in Week 1 so all members can develop and test in parallel.
Merge Conflicts	Strict separation of concerns: one owner per file, small feature branches, and bi-weekly integration merges.